



CHAPTER 6

텍스트 요약, 텍스트 생성, 그리고 벡터 표현

긴 글을 어떻게 요약하고 새로운 문장을 생성하는가?

학습 목표

- 핵심 질문 : 텍스트를 요약하고 새로 생성하는 방법은 무엇인가?
 - 텍스트 요약 : 추출적(extractive) 요약과 생성적(abstractive) 요약의 원리를 구분하고 설명할 수 있다
 - 텍스트 생성 : 마르코프 체인과 n-gram 언어 모델로 텍스트를 생성하고, 신경망 기반 생성과 비교할 수 있다
 - 벡터 표현 : 원핫 인코딩, BoW/TF-IDF, 단어 임베딩, 문서 벡터의 차이를 이해하고 구현할 수 있다
 - TextRank 알고리즘을 직접 구현하여 실제 기사 데이터를 요약할 수 있다
 - ROUGE 등 요약 평가 지표로 요약 결과의 품질을 정량적으로 평가할 수 있다

강의 개요

◦ 이번 주차 구성

- 6.1 문서 요약 : 자동 요약의 개념, 추출적/추상적 요약, TextRank 구현, 요약 평가
- 6.2 텍스트 생성 : 마르코프 체인, n-gram 언어 모델, 신경망 언어 모델, 디코딩 전략
- 6.3 벡터 표현 : 인코딩, 원핫 인코딩, BoW/TF-IDF, 단어 임베딩, 문서 벡터
- 6.5 요약 및 참고 문헌

◦ 선수 지식

- 파이썬 기초, NumPy/Pandas 기본 사용법, 토큰화와 불용어 제거 (3~5주차 내용)

6.1 문서 요약 (Text Summarization)

- **학습 목표 :**

- 자동 텍스트 요약이란?
- 텍스트 요약의 고수준 뷰(목적 / 입력 / 출력)
- 추출적 요약 : 단어 빈도, TF-IDF, 문장 유사도, 클러스터링
- 추상적(생성적) 요약 : 시퀀스 투 시퀀스, 인코더-디코더, 어텐션, 트랜스포머
- TextRank 알고리즘의 원리와 파이썬 구현
- 요약 품질 평가 : ROUGE, BLEU, BERTScore

6.1 문서 요약 (Text Summarization)

- 소개 :

- 텍스트 데이터를 처리할 때 겪게 되는 한 가지 큰 어려움은 데이터가 클 수 있고 문서, 이메일, 웹 페이지와 같은 다양한 형태로 올 수 있다는 점임
- 이러한 자료를 읽고 이해하는 것은 번거로운 일임
- 또한 사람들은 엄청난 양의 정보를 읽을 때 인내심이 줄어드는 경향이 있음
- 사람들은 '한 입 크기'로 정보를 소비하는 것을 선호함
- 사람들이 읽어야 하는 콘텐츠의 양을 줄이면서 중심 아이디어를 그대로 유지할 필요가 있음
- 이를 돕기 위해 콘텐츠 제공업체들은 사용자에게 정보의 요지를 제공하는 텍스트 요약 기능을 제공함
- 비즈니스적으로도 텍스트 요약 자동화에 대한 수요가 큼(뉴스 브리핑, 고객 리뷰 요약, 회의록 요약 등)

6.1 문서 요약 (Text Summarization)

◦ 자동 텍스트 요약이란? :

- 자동 텍스트 요약이란 원래 콘텐츠에 존재하는 모든 핵심 정보를 보존하는, 텍스트의 간결한 버전을 생성하는 과정임
- 콘텐츠 제공업체들은 독서 습관 변화에 적응해 왔으며, 짧은 기사와 게시물을 보는 것이 매우 일반적임
- 다른 주요 적용 사례는 내용에 대한 요약과 읽는 데 걸리는 시간 추정치를 함께 제공하는 것임
- 이러한 기능들은 NLP를 통해 개발했으며, 텍스트 요약 관련 기술과 도구는 지속적으로 발전하고 있음
- 압축률(compression ratio) : 요약문 길이 / 원문 길이. 일반적으로 10~30% 수준을 목표로 함

6.1 문서 요약 (Text Summarization)

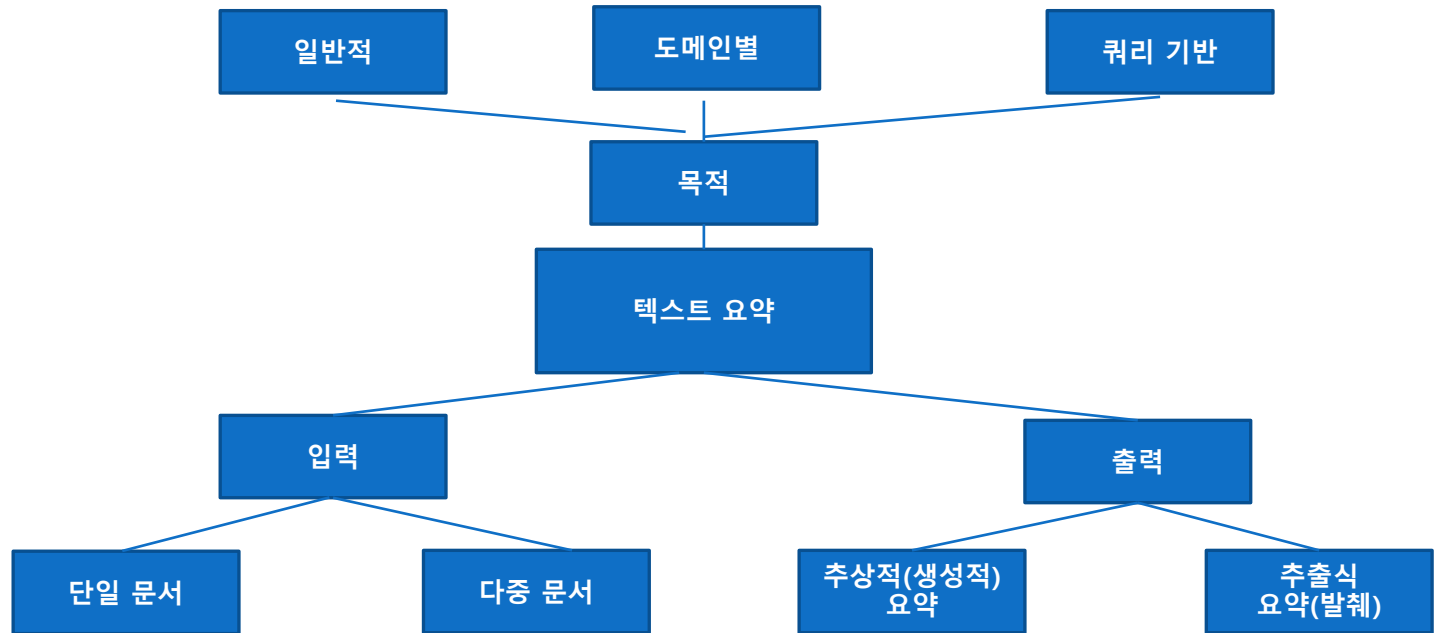
◦ 자동 텍스트 요약의 이점 :

- 샘플링 : 자동 요약은 기사를 실제로 공개하기에 앞서 문서나 기사의 샘플이 될 수 있음, 이 샘플을 통해 전체 기사를 읽을 지 말지를 결정할 수 있음
- 검색 : 요약은 검색에 도움을 줄 수 있음, 사용자가 선택할 수 있도록 검색 결과에 표시할 수 있음
- 인덱싱 : 원본 문서의 압축적 표현으로써 검색 색인에 요약본을 사용할 수 있음, 이는 필요한 스토리지 양을 줄이는 데도 도움이 됨
- 읽기 시간 : 요약을 잘 구성하면 실제 문서 대신 사용할 수 있음, 훨씬 더 짧은 시간에 읽을 수 있음
- 질의 응답 : 챗봇은 사용자 질문에 대한 응답으로 자동 요약을 제공할 수 있음

6.1 문서 요약 (Text Summarization)

- 텍스트 요약의 고수준 뷰

- 텍스트 요약의 주제를 다양한 각도(목적 / 입력 / 출력)에서 볼 수 있음
- 다음 장에서는 텍스트 요약의 각 측면에 대해서 살펴보고자 함



6.1 문서 요약 (Text Summarization)

- 텍스트 요약의 고수준 뷰 (계속) :

- 목적 :

- 텍스트 요약의 주제를 살펴볼 수 있는 다양한 각도가 있음
 - 사람들이 일반적으로 무엇을 위해 요약을 사용하는지 아는 것으로 텍스트 요약의 목적을 확인할 수 있음
 - 세 가지 종류 : 일반적 요약, 도메인 한정 요약, 쿼리 기반 요약
 - 일반적 요약 : 텍스트 입력이 충분한 길이를 가진다면 다양한 도메인의 소스를 적절히 요약할 수 있음
 - 도메인 한정 요약 : 금융, 쇼핑, 의료, 여행과 같은 도메인에 한정될 수 있음, 해당 도메인에서 중요한 특정 용어를 고려해야 함
 - 쿼리 기반 요약 : 검색 창이나 챗봇에서 사용자 입력에 대한 응답을 제공하는 데 중점을 두므로 질문과의 관련성이 핵심임

6.1 문서 요약 (Text Summarization)

- 텍스트 요약의 고수준 뷰 (계속) :

- 입력 :

- 텍스트 요약에 대한 입력 소스는 단일 문서 혹은 여러 문서일 수 있음
 - 단일 문서 : 기사, 이메일, 소셜 미디어 게시물, 긴 워드 문서와 같이 많은 내용을 포함하는 하나의 문서, 사용자 중심 요약 소프트웨어가 작동하는 전형적인 모드임
 - 여러 문서 : 여러 문서를 요약하고 하나의 요약을 만드는 데도 사용할 수 있음, 여러 문서를 분석하는 과정에서 시간을 절약해야 하는 기업에 유용함
 - 예를 들어, 법률 회사는 여러 판례 문서를 하나의 요약으로 간추리는 프로세스를 실행할 수 있음

6.1 문서 요약 (Text Summarization)

- 출력 : 추출적 요약 vs 추상적 요약

- 추출적(extractive) 요약

- 원문에서 중요한 문장이나 구를 '그대로 골라서' 이어 붙임
 - 장점 : 구현이 쉽고, 문법 오류와 사실 왜곡이 거의 없음
 - 단점 : 문장 간 연결이 어색할 수 있고, 원문에 없는 표현으로 압축하지 못함

- 추상적(abstractive, 생성적) 요약

- 원문의 의미를 이해한 뒤 '새로운 문장'을 만들어 냄
 - 장점 : 사람이 쓴 요약과 유사하고 압축률이 높음
 - 단점 : 학습이 어렵고, 환각(hallucination)으로 사실과 다른 내용을 생성할 위험이 있음

6.1 문서 요약 (Text Summarization)

◦ 추출적 텍스트 요약 :

- 원문을 검토한 후, 너무 많은 의미를 잃지 않고 텍스트에 포함된 아이디어를 가장 잘 전달하는 문장이나 구를 추출함
- 추출적 요약을 수행하는 알고리즘은 다양하지만 일반적으로 같은 원리로 동작함
- 핵심 원리 : 텍스트의 문장들을 중요도별로 순위를 매기는 것
- 문장이 텍스트 문서의 전체적 의미에 얼마나 기여하는지에 따라 문장의 중요도를 해석할 수 있음
- 순위를 매긴 후 가장 높은 순위를 얻은 문장들을 골라 요약을 구성함

6.1 문서 요약 (Text Summarization)

- 추출적 텍스트 요약 (계속) : 문장 순위 기준

- 단어 빈도(word frequency)

- 문장의 단어가 문서에서 얼마나 자주 나타나는지에 따라 문장의 순위를 정함
 - 자주 사용되는 단어가 포함된 문장이 전체 문서의 전반적인 의미를 나타낸다는 아이디어
 - 단어 빈도로 순위를 매길 때는 불용어처럼 문서를 제대로 대표하지 않는 단어들을 무시해야 함

- 문장 유사도(sentence similarity)

- 주어진 문장과 문서 안의 다른 문장들이 얼마나 유사한지를 나타내는 척도
 - 많은 문장과 유사한 문장을 선택하면 문서의 많은 의미를 대표할 수 있음

- 클러스터링 중심(clustering centroid)

- 문서의 문장들을 그룹으로 묶고, 각 클러스터의 중심에 가장 가까운 문장을 선택함

6.1 문서 요약 (Text Summarization)

◦ 추출적 요약의 가중치 : TF-IDF

- 단순 빈도(TF)만 쓰면 '그리고', 'the' 같은 흔한 단어가 과대평가됨
- TF-IDF는 '한 문서에는 자주 나오지만 다른 문서에는 드물게 나오는 단어'에 큰 가중치를 줌
 - $TF(t, d) = \text{문서 } d \text{에서 단어 } t \text{의 상대 빈도} = (\text{등장 횟수}) / (\text{문서 } d \text{의 총 단어 수})$
 - ※ 5주차는 TF를 원시 빈도(등장 횟수)로 정의하므로 값의 스케일이 다름
 - $IDF(t) = \log(N / (1 + df(t))) + 1$ (음수 방지 보정항), $N = \text{전체 문서 수}$, $df(t) = t \text{가 등장한 문서 수}$
 - ※ 5주차와 scikit-learn 기본값은 $\log((1+N)/(1+df(t))) + 1$ 형태를 사용함
 - $TF-IDF(t, d) = TF(t, d) \times IDF(t)$
- 문장 점수 = 문장에 포함된 단어들의 TF-IDF 합 / 문장 길이(정규화)
- 구현 : scikit-learn 의 TfidfVectorizer 한 줄로 문서-단어 행렬을 만들 수 있음

6.1 문서 요약 (Text Summarization)

- 문장 유사도 : 코사인 유사도

- 두 문장을 벡터 u, v 로 표현했을 때 유사도는 두 벡터가 이루는 각도로 측정함
 - $\cos(u, v) = (u \cdot v) / (\|u\| \times \|v\|)$
 - 값의 범위는 $-1 \sim 1$, 임베딩이 모두 양수인 경우 $0 \sim 1$
- 왜 유클리드 거리가 아니라 코사인인가?
 - 문장 길이가 길어지면 벡터의 크기(norm)가 커지지만 '내용'은 같을 수 있음
 - 코사인 유사도는 크기를 정규화하므로 길이의 영향을 받지 않음
- n 개 문장이면 $n \times n$ 유사도 행렬(similarity matrix)을 만들 수 있고, 이것이 TextRank 그래프의 가중치가 됨

6.1 문서 요약 (Text Summarization)

- 추상적(생성적) 텍스트 요약 :

- 추상적(생성적) 요약 : 원문의 문장을 그대로 옮기는 대신 의미를 재구성해 새로운 문장을 생성함 (원문 어휘를 재사용할 수 있음)
- 추상화는 본질적으로 원문을 좀 더 간결한 형태로 '다시 쓰는' 것임
- 원래 텍스트의 의미를 학습한 후 요약을 만들어야 하므로 알고리즘을 만들기가 훨씬 어려움
- 따라서 추상적 텍스트 요약은 보통 신경망 모델과 같이 더 진보된 기술로 이루어짐
- 대표적인 두 가지 구성 요소 : 시퀀스 투 시퀀스(seq2seq), 인코더-디코더

6.1 문서 요약 (Text Summarization)

◦ 추상적 텍스트 요약 : 시퀀스 투 시퀀스

- 추상적 텍스트 요약의 목표는 입력 텍스트를 가져와서 그 텍스트에 대한 추상적인 요약을 생성하는 것
- 입력 텍스트는 단어나 문자들의 시퀀스로 간주할 수 있으며, 요약은 출력 시퀀스로 간주할 수 있음
- 따라서 텍스트 요약은 하나의 시퀀스를 다른 시퀀스로 변환하는 시퀀스 투 시퀀스 문제로 볼 수 있음
- 시퀀스 투 시퀀스 신경망은 일반적으로 기계 번역이나 질의응답에 사용함
- 하지만 입력으로부터 더 짧은 출력 시퀀스를 만드는 것이 목적인 텍스트 요약에도 사용할 수 있음

6.1 문서 요약 (Text Summarization)

◦ 추상적 텍스트 요약 : 인코더 - 디코더

- 시퀀스 투 시퀀스 변환을 수행하는 방법은 인코더-디코더 네트워크를 사용하는 것임
 - 인코더 : 입력 단어 시퀀스를 받아 내재적인 표현(문맥 벡터)을 만듦
 - 디코더 : 입력 표현을 받아 단어들의 출력 시퀀스를 만듦
- 초기 구조의 한계 : 입력 문장이 길어지면 하나의 고정 길이 문맥 벡터에 모든 정보를 담기 어려움(정보 병목)
- 해결책 : 어텐션(attention) 메커니즘 - 디코더가 출력 단어를 만들 때마다 입력의 어느 부분을 볼지 선택함

6.1 문서 요약 (Text Summarization)

- 어텐션과 트랜스포머

- 어텐션(Attention)

- $\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$
 - 질의(Query)와 키(Key)의 유사도로 가중치를 만들고, 값(Value)을 가중합함

- 트랜스포머(Transformer, 2017)

- 순환 구조(RNN) 없이 셀프 어텐션만으로 문맥을 파악, 병렬 학습이 가능해 대규모 학습에 유리
 - 인코더-디코더 구조를 그대로 유지하면서 어텐션을 핵심 연산으로 사용

- 오늘날 추상적 요약의 사실상 표준 구조이며, 6.3절의 문맥 임베딩과도 직접 연결됨

6.1 문서 요약 (Text Summarization)

◦ 사전 학습 요약 모델 (실무 관점)

- BART : 원문을 일부 손상시킨 뒤 복원하도록 사전 학습한 인코더-디코더 모델, 요약에 강함
- T5 : 모든 NLP 과제를 'text-to-text'로 통일한 모델, 'summarize: ...' 형태의 프롬프트를 사용
- PEGASUS : 문서에서 중요한 문장을 통째로 가린 뒤 복원하도록 학습 - 요약에 특화된 사전 학습
- 한국어 : KoBART, KoT5 등의 공개 모델을 사용할 수 있음

◦ 허깅페이스로 3줄 요약

```
from transformers import pipeline
summarizer = pipeline('summarization',
    model='facebook/bart-large-cnn')
summarizer(text, max_length=120, min_length=40,
    do_sample=False)
```

6.1 문서 요약 (Text Summarization)

- **TextRank :**

- 추출적 텍스트 요약에 널리 사용되는 알고리즘 중 하나
- PageRank 에 기반을 두고 만들어짐
- PageRank 는 구글이 검색 결과를 정렬할 때 가장 먼저 사용한 알고리즘이며, 단순한 링크 개수가 아니라 그 페이지를 참조한 페이지들의 중요도를 가중해 재귀적으로 순위를 정하는 원칙에 따라 동작함
- 마찬가지로 TextRank 는 주어진 문장과 다른 문장이 얼마나 유사한지에 따라 문장의 순위를 결정함
- 비지도(unsupervised) 방식이므로 정답 요약 데이터가 없어도 바로 적용할 수 있는 것이 큰 장점임

6.1 문서 요약 (Text Summarization)

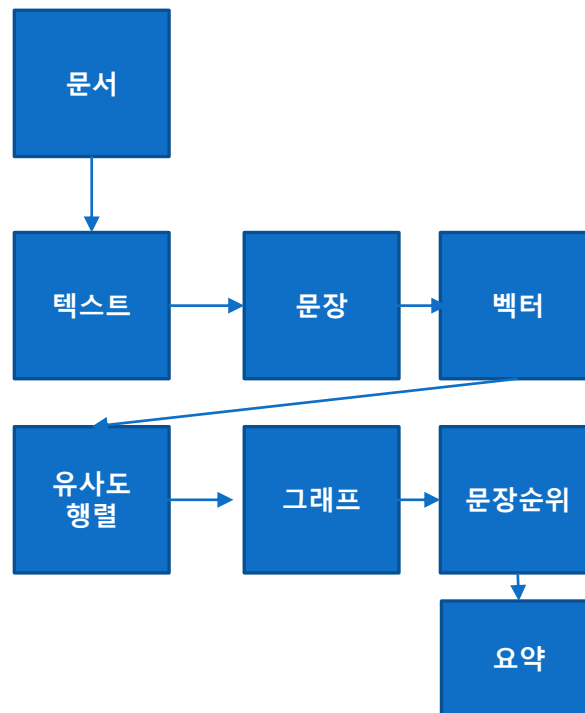
- **TextRank 알고리즘의 동작 :**

- 1. 문서에서 텍스트를 읽고 추출함
- 2. 텍스트를 문장으로 나눔
- 3. 문장의 각 단어를 벡터로 변환함(GloVe 등 사전 학습 단어 벡터 사용)
- 4. 단어 벡터의 평균으로 문장 전체의 벡터를 구함
- 5. 문장 사이의 유사도 행렬을 계산함(코사인 유사도)
- 6. 유사도 행렬을 인접 행렬로 삼아 가중 그래프를 생성함
- 7. 그래프에 PageRank 를 적용해 문장의 중요도 점수를 구함
- 8. 점수가 높은 상위 n개 문장을 원문 순서대로 배열해 요약을 만
들

6.1 문서 요약 (Text Summarization)

- **TextRank :**

- TextRank 과정에서의 데이터 흐름
- 문서 → 텍스트 → 문장 → 벡터 → 유사도 행렬 → 그래프 → 문장 순위 → 요약



6.1 문서 요약 (Text Summarization)

- **TextRank 점수 계산식**

- PageRank 를 가중 그래프로 확장한 형태
 - $WS(V_i) = (1 - d)/n + d \times \sum_j [w_{ij} / \sum_k w_{jk}] \times WS(V_j)$
(원논문은 $(1-d)$, networkx는 $(1-d)/n$)
 - V_i : 문장 i 를 나타내는 노드, w_{ij} : 문장 i 와 j 의 코사인 유사도
 - d : 감쇠 계수(damping factor), 보통 0.85 를 사용
- 해석 : '중요한 문장과 많이 닮은 문장'이 다시 중요해지는 재귀적 정의
- 계산 : 점수 벡터가 수렴할 때까지 반복(멱승법), networkx 의 pagerank() 가 이를 수행함

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (1) : 라이브러리 준비**

- 데이터셋을 가져온 후 TextRank 알고리즘을 이용해 포함된 기사를 요약함
- 1. 관련 라이브러리를 추가함

```
import numpy as np
import pandas as pd
import nltk
import re
import networkx as nx
from sklearn.metrics.pairwise import cosine_similarity

# NLTK 3.9 이상 필수 (3.8.1 이하는 punkt)
nltk.download('punkt')
nltk.download('punkt_tab')
nltk.download('stopwords')
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (2) : GloVe 단어 벡터 준비**
 - 2. GloVe 단어 벡터를 준비함 (없으면 자동으로 내려받음)
 - 3. 텍스트 파일을 사전(dict)으로 적재해 단어 벡터를 조회함

```
import os, numpy as np
import gensim.downloader as api          # pip install gensim
GLOVE_DIR = 'data/glove/'
GLOVE_TXT = GLOVE_DIR + 'glove.6B.50d.txt'
os.makedirs(GLOVE_DIR, exist_ok=True)
if not os.path.exists(GLOVE_TXT):        # 없으면 자동 내려받음(약 66MB)
    api.load('glove-wiki-gigaword-50').save_word2vec_format(
        GLOVE_TXT, write_header=False)

def load_glove_vectors(fn):
    model = {}
    with open(fn, encoding='utf-8') as f:
        for line in f:
            parts = line.split()
            model[parts[0]] = np.array(parts[1:], dtype=float)
    return model
glove_vectors = load_glove_vectors(GLOVE_TXT)
```

6.1 문서 요약 (Text Summarization)

- TextRank 실습 (3) : 데이터 불러오기

- 4. data 디렉터리의 CSV 파일을 불러옴 — 파일이 없으면 다음 장의 예시 코퍼스를 사용함

```
import os
CSV_PATH = 'data/tennis_articles_v4.csv'
articles = (pd.read_csv(CSV_PATH) if os.path.exists(CSV_PATH)
```

article_id		article_text	source
0	1	Maria Sharapova has basically no friends as tennis players on the WTA Tour. The Russian player has no problems in openly speaking about it and in a recent interview she said: 'I don't really hide any feelings too much. I think everyone knows this is my job here. When I'm on the courts or when I'm on the court playing, I'm a competitor and I want to beat every single person whether they're in the locker room or across the nets. So I'm not the one to strike up a conversation about the weather and know that in the next few minutes I have to go and try to win a tennis match. I'm a pretty competitive girl. I say my hellos, but I'm not sending any players flowers as well. Umm, I'm not really friendly or close to many players. I have not a lot of friends away from the courts.' "When she said she is not really close to a lot of players, is that something strategic that she is doing? Is it different on the men's tour than the women's tour? 'No, not at all. I think just because you're in the sa...	https://www.tennisworldusa.org/tennis/news/Maria_Sharapova/62220/i-do-not-have-friends-in-tennis-says-maria-sharapova/
1	2	BASEL, Switzerland (AP), Roger Federer advanced to the 14th Swiss Indoors final of his career by beating seventh-seeded Daniil Medvedev 6-1, 6-4 on Saturday. Seeking a ninth title at his hometown event, and a 99th overall, Federer will play 93th-ranked Marius Copil on Sunday. Federer dominated the 20th-ranked Medvedev and had his first match-point chance to break serve again at 5-1. He then dropped his serve to love, and let another match point slip in Medvedev's next service game by netting a backhand. He clinched on his fourth chance when Medvedev netted from the baseline. Copil upset expectations of a Federer final against Alexander Zverev in a 6-3, 6-7 (6), 6-4 win over the fifth-ranked German in the earlier semifinal. The Romanian aims for a first title after arriving at Basel without a career win over a top-10 opponent. Copil has two after also beating No. 6 Marin Cilic in the second round. Copil fired 26 aces past Zverev and never dropped serve, clinching after 2 1/2 hours w...	http://www.tennis.com/pro-game/2018/10/copl-stuns-5th-ranked-zverev-to-reach-swiss-indoors-final/77721/

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (4) : 예시 코퍼스**

- tennis_articles_v4.csv 가 없을 때 아래 예시 기사로 대체함
- 기사마다 문장이 3 개 이상이면 TextRank 가 정상 동작함

```
SAMPLE_ARTICLES = [  
    "Maria won the final in straight sets. She said the crowd gave "  
    "her energy. It was her first title of the season. Her coach "  
    "praised her serve. She will play in Rome next week.",  
    "Roger will skip the clay season. He wants to protect his knee. "  
    "The decision surprised many fans. His team said the schedule "  
    "was too heavy. He plans to return on grass in June.",  
    "The tour introduced a new tie-break rule. Players welcomed it. "  
    "Officials said matches had become too long. The rule applies "  
    "to the final set. Fans will see shorter matches from now on.",  
]
```

```
articles = pd.DataFrame(  
    {'article_id': range(1, len(SAMPLE_ARTICLES) + 1),  
    'article_text': SAMPLE_ARTICLES})  
print(articles.shape)
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (5) : 전처리 함수**

- 5. NLTK 영어 불용어를 내려받은 후 불러옴
- 6. 기사 텍스트를 정제하고 단어.문장으로 토큰화하는 함수들을 만듦

```
from nltk.corpus import stopwords
from nltk.tokenize import sent_tokenize, \
    word_tokenize
stop_words = stopwords.words('english')
CLEAN_PATTERN = r'^a-zA-Z\s]'

def clean(word):
    return re.sub(CLEAN_PATTERN, '', word)

def clean_sentence(sentence):
    words = [clean(w) for w in sentence]
    return [w for w in words if w]

def clean_sentences(sentences):
    return [clean_sentence(s) for s in sentences]
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (6) : 전처리 함수 (계속)**

- 토큰화 → 소문자 변환 → 불용어 제거 → 특수문자 제거 순으로 연결해 적용함
- 원본 교재의 코드에는 오타가 있어 아래와 같이 수정하여 사용함

```
def lower(sentences):  
    return [[w.lower() for w in s]  
            for s in sentences]  
  
def remove_stopwords(sentences):  
    return [[w for w in s  
            if w not in stop_words]  
            for s in sentences]  
  
def tokenize_words(sentences):  
    return [word_tokenize(s) for s in sentences]
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (7) : 기사에 함수 적용**

- 7. 각 기사를 문장으로 나누고, 각 문장을 단어로 나눔
- 데이터프레임을 사용하고 있으므로 데이터셋의 기사 각각에 함수를 적용함

```
articles['SentencesInArticle'] = \
    articles.article_text.apply(sent_tokenize)
```

```
articles['WordsInSentences'] = (
    articles.SentencesInArticle
    .apply(tokenize_words)
    .apply(lower)
    .apply(remove_stopwords)
    .apply(clean_sentences))
```

6.1 문서 요약 (Text Summarization)

- TextRank 실습 (8) : 중간 결과 확인
- 8. SentencesInArticle 과 WordsInSentences 열만 골라 결과를 확인함

```
articles = articles[['SentencesInArticle', 'WordsInSentences']]
```

SentencesInArticle		WordsInSentences
0	0	Maria Sharapova has basically no friends as tennis players on the WTA Tour. The Russian player has no problems in openly speaking about it and in a recent interview she said: 'I don't really hide any feelings too much. I think everyone knows this is my job here. When I'm on the courts or when I'm on the court playing, I'm a competitor and I want to beat every single person whether they're in the locker room or across the nets. So I'm not the one to strike up a conversation about the weather and know that in the next few minutes I have to go and try to win a tennis match. I'm a pretty competitive girl. I say my hellos, but I'm not sending any players flowers as well. Umm, I'm not really friendly or close to many players. I have not a lot of friends away from the courts.' "When she said she is not really close to a lot of players, is that something strategic that she is doing? Is it different on the men's tour than the women's tour?" "No, not at all. I think just because you're in the sa...
1	1	[BASEL, Switzerland (AP), Roger Federer advanced to the 14th Swiss Indoors final of his career by beating seventh-seeded Daniil Medvedev 6-1, 6-4 on Saturday. Seeking a ninth title at his hometown event, and a 99th overall, Federer will play 93th-ranked Marius Copil on Sunday. Federer dominated the 20th-ranked Medvedev and had his first match-point chance to break serve again at 5-1. He then dropped his serve to love, and let another match point slip in Medvedev's next service game by netting a backhand. He clinched on his fourth chance when Medvedev netted from the baseline. Copil upset expectations of a Federer final against Alexander Zverev in a 6-3, 6-7 (6), 6-4 win over the fifth-ranked German in the earlier semifinal. The Romanian aims for a first title after arriving at Basel without a career win over a top-10 opponent. Copil has two after also beating No. 6 Marin Cilic in the second round. Copil fired 26 aces past Zverev and never dropped serve, clinching after 2 1/2 hours w...

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (9) : 문장 벡터 만들기**

- 9. 문장에 포함된 단어 벡터들의 평균으로 문장 벡터를 만듦
- 사전에 없는 단어(OOV)는 영벡터로 처리함

```
VECTOR_SIZE = 50
EMPTY_VECTOR = np.zeros(VECTOR_SIZE)

def sentence_vector(sentence):
    if len(sentence) == 0:
        return EMPTY_VECTOR
    vecs = [glove_vectors.get(w, EMPTY_VECTOR)
             for w in sentence]
    return sum(vecs) / len(sentence)

def sentences_to_vectors(sentences):
    return [sentence_vector(s) for s in sentences]
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (10) : 유사도 행렬**

- 10. SentenceVector 열을 생성함
- 11. 유사도 행렬은 한 기사 안의 문장이 서로 얼마나 유사한지를 포착함

```
articles['SentenceVector'] = (  
    articles.WordsInSentences  
    .apply(sentences_to_vectors))  
  
def similarity_matrix(sentence_vectors):  
    n = len(sentence_vectors)  
    sim_mat = np.zeros([n, n])  
    for i in range(n):  
        for j in range(n):  
            if i == j:  
                continue  
            a = sentence_vectors[i].reshape(1, -1)  
            b = sentence_vectors[j].reshape(1, -1)  
            sim_mat[i][j] = max(0.0, cosine_similarity(a, b)[0, 0])  
    return sim_mat
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (11) : 그래프와 PageRank**

- 12. 각 기사에 대해 유사도 행렬을 생성함
- 13. 유사도 행렬을 인접 행렬로 삼아 networkx 그래프를 만들고 PageRank 를 적용함
- 그래프는 각 문장의 상대적 중요도를 '다른 문장들과 얼마나 비슷한가'로 결정함

```
articles['SimMatrix'] = (  
    articles.SentenceVector  
    .apply(similarity_matrix))
```

```
import networkx as nx
```

```
def compute_graph(sim_matrix):  
    nx_graph = nx.from_numpy_array(sim_matrix)  
    scores = nx.pagerank(nx_graph)  
    return scores
```

```
articles['Graph'] = (  
    articles.SimMatrix.apply(compute_graph))
```

6.1 문서 요약 (Text Summarization)

- **TextRank 실습 (12) : 상위 문장 선택과 요약**

- 14. 점수 상위 n개 문장을 골라 원문 순서로 이어 붙이는 함수를 작성함

```
def get_ranked_sentences(sentences, scores, n=3):  
    top = sorted(range(len(sentences)), key=lambda i: -  
        scores[i])[:n]  
    # 읽기 쉽도록 원문 순서로 되돌린다  
    return ' '.join(sentences[i] for i in sorted(top))
```

- 15. 데이터프레임에 적용해 Summary 열을 생성하고 결과를 확인함

```
articles['Summary'] = articles.apply(  

```

```
[maria, sharapova, has, basically, no, friends, as, tennis, players, on, the, wta, tour],  
[the, russian, player, has, no, problems, in, openly, speaking, about, it, and, in, a,  
recent, interview, she, said], [i, don't, really, hide, any, feelings, too, much, i, think,  
everyone, knows, this, is, my, job, here]...
```

6.1 문서 요약 (Text Summarization)

- **요약 품질은 어떻게 평가하는가?**

- 요약에는 '정답'이 하나로 정해지지 않음 - 사람마다 좋은 요약이 다를 수 있음
- 평가 방법은 크게 두 가지
 - 내재적(intrinsic) 평가 : 사람이 작성한 참조 요약과 얼마나 겹치는지 측정(ROUGE, BLEU)
 - 외재적(extrinsic) 평가 : 요약을 사용한 후속 작업(검색, 분류)의 성능으로 간접 측정
- 정량 지표는 편리하지만 '의미가 같은 다른 표현'을 인정하지 못하는 한계가 있음
- 따라서 정량 지표 + 사람 평가(정확성, 일관성, 가독성)를 함께 사용하는 것이 표준임

6.1 문서 요약 (Text Summarization)

- **ROUGE : 요약 평가의 표준 지표**

- ROUGE = Recall-Oriented Understudy for Gisting Evaluation
- 참조 요약(reference)과 생성 요약(candidate) 사이의 n-gram 중복을 측정함
 - $\text{ROUGE-N Recall} = (\text{겹치는 n-gram 수}) / (\text{참조 요약의 n-gram 수})$
 - $\text{ROUGE-N Precision} = (\text{겹치는 n-gram 수}) / (\text{생성 요약의 n-gram 수})$
 - $\text{F1} = 2PR / (P + R)$
- 주요 변형
 - ROUGE-1 : 단어(unigram) 중복 - 내용어를 잘 담았는가
 - ROUGE-2 : 2-gram 중복 - 표현의 자연스러움/어순
 - ROUGE-L : 최장 공통 부분 수열(LCS) 기반 - 문장 구조의 유사성

6.1 문서 요약 (Text Summarization)

- ROUGE 계산 예시

- 참조 요약 : the cat was found under the bed
- 생성 요약 : the cat was under the bed
 - 참조 unigram 7개, 생성 unigram 6개, 겹치는 unigram 6개
 - ROUGE-1 Recall = $6 / 7 = 0.857$
 - ROUGE-1 Precision = $6 / 6 = 1.000$
 - ROUGE-1 F1 = $2 \times (0.857 \times 1.0) / (0.857 + 1.0) = 0.923$

- 파이썬 구현

```
from rouge_score import rouge_scorer
scorer = rouge_scorer.RougeScorer(['rouge1', 'rouge2', 'rougeL'],
    use_stemmer=True)
reference = 'the cat was found under the bed'      # 참조 요약
candidate = 'the cat was under the bed'           # 생성 요약
print(scorer.score(reference, candidate))
```

6.1 ROUGE-L 계산

- **ROUGE-L : 최장 공통 부분 수열(LCS) 기반 — 어순을 반영한 요약 평가 지표**
 - LCS 는 연속일 필요가 없고 순서만 유지되면 되므로 문장 구조 유사성을 본다
- **공식**
 - $R_{lcs} = LCS(\text{참조}, \text{생성}) / |\text{참조}|$ $P_{lcs} = LCS(\text{참조}, \text{생성}) / |\text{생성}|$
 - $F_{lcs} = (1 + \beta^2) \cdot R_{lcs} \cdot P_{lcs} / (R_{lcs} + \beta^2 \cdot P_{lcs})$, 보통 $\beta = 1$
- **예시 (ROUGE-1 예시와 같은 문장)**
 - 참조 : the cat was found under the bed (7 단어)
 - 생성 : the cat was under the bed (6 단어)
 - LCS = the cat was under the bed → 길이 6
 - $R_{lcs} = 6 / 7 = 0.857$, $P_{lcs} = 6 / 6 = 1.000$
 - $F_{lcs} = 2 \times 0.857 \times 1.000 / (0.857 + 1.000) = 0.923$
- **ROUGE-1 과 값이 같아 보이지만, 어순이 뒤바뀐 요약에서는 ROUGE-L 만 점수가 떨어진다**

6.1 문서 요약 (Text Summarization)

◦ 그 밖의 평가 지표

- BLEU : 정밀도 중심 지표. 기계 번역에서 유래했으며 짧은 출력에 대한 길이 페널티(BP)를 둠
 - 요약은 '빠짐없이 담았는가'가 중요하므로 재현율 중심의 ROUGE 를 더 선호함
- METEOR : 동의어와 어간(stem)까지 매칭해 표현 다양성을 일부 인정함
- BERTScore : 단어를 문맥 임베딩으로 바꾼 뒤 코사인 유사도로 매칭 - 표현이 달라도 의미가 같으면 높은 점수
- 사실성(faithfulness) 평가 : 생성 요약이 원문에 없는 내용을 지어내지 않았는지 별도로 확인해야 함

6.1 문서 요약 (Text Summarization)

◦ 6.1 정리 : 언제 무엇을 쓸 것인가

- 정답 요약 데이터가 없다 → TextRank 등 비지도 추출 요약
- 도메인 데이터가 충분하다 → BART/T5 계열 미세조정(추상 요약)
- 사실 왜곡이 절대 안 된다(법률, 의료) → 추출 요약 또는 추출+생성 하이브리드
- 문서가 매우 길다 → 청크 단위로 나눠 요약 후 다시 요약(map-reduce 방식)
- 평가 → ROUGE-1/2/L 로 정량 측정 + 사람 평가로 사실성 확인

6.2 텍스트 생성 (Text Generation)

- **학습 목표 :**

- 마르코프 체인의 개념과 텍스트 생성에의 적용
- n-gram 언어 모델과 확률 추정
- 마르코프 체인을 이용한 텍스트 생성 구현
- 신경망 언어 모델(RNN, Transformer/GPT)로의 발전 과정
- 디코딩 전략 : greedy, beam search, top-k, top-p, temperature
- 생성 텍스트의 위험 : 환각, 반복, 편향

6.2 텍스트 생성 (Text Generation)

- **마르코프 체인을 사용한 텍스트 생성**

- 텍스트 생성의 핵심 개념은 텍스트의 통계적인 속성을 활용하는 것임
- 이것은 출력 텍스트를 더 사실적으로 만듦
- 텍스트에 내재된 흥미로운 패턴을 식별하기 위해 이를 재배열할 수 있다면 텍스트 자체에 대한 정보를 얻을 수 있음
- 텍스트는 다양한 방법으로 생성할 수 있음
- 먼저, 가장 단순한 통계적 방법인 마르코프 체인을 사용해 텍스트를 생성해 봄

6.2 텍스트 생성 (Text Generation)

◦ 마르코프 체인

- 마르코프 체인은 확률에 기반하는 상태 전환에 대한 수학적인 시스템임
- 어떤 시스템이 어떻게 현재 상태에 도달했든지 상관없이 가능한 미래 상태는 모두 사전에 정해져 있음, 또한 다음 상태로의 이동은 일련의 확률에 기초함
- 마르코프 가정 : $P(w_t \mid w_1, \dots, w_{(t-1)}) \approx P(w_t \mid w_{(t-1)})$
- 마르코프 체인은 시점 단위로 작동함, 각 시점 후에 시스템은 새로운 상태를 선택함
- 마르코프 체인은 금융, 게임 이론, 경제학 같은 분야에서 유용하며, NLP에서는 텍스트 자원의 통계 패턴으로 상태 전환 확률을 만듦
- 여기서 하나의 통계적 속성은 '주어진 단어 뒤에 다른 단어가 올 확률'임

6.2 텍스트 생성 (Text Generation)

- 상태 전이 확률 : 간단한 예

- 코퍼스 : '나는 밥을 먹었다 / 나는 빵을 먹었다 / 나는 밥을 남겼다'
- '나는' 다음에 오는 단어의 빈도 : 밥을 2회, 빵을 1회
 - $P(\text{밥을} \mid \text{나는}) = 2/3 \approx 0.67$
 - $P(\text{빵을} \mid \text{나는}) = 1/3 \approx 0.33$
- '밥을' 다음에 오는 단어의 빈도 : 먹었다 1회, 남겼다 1회
 - $P(\text{먹었다} \mid \text{밥을}) = 0.5$, $P(\text{남겼다} \mid \text{밥을}) = 0.5$
- 생성 : 시작 단어를 고른 뒤 위 확률에 따라 다음 단어를 무작위로 뽑는 과정을 반복함

6.2 텍스트 생성 (Text Generation)

- **n-gram 언어 모델**

- 언어 모델(Language Model) : 단어 시퀀스의 확률 $P(w_1, w_2, \dots, w_n)$ 을 계산하는 모델
- 연쇄 법칙 : $P(w_1..w_n) = P(w_1) \times P(w_2|w_1) \times \dots \times P(w_n | w_1..w_{(n-1)})$
- n-gram 근사 : 직전 (n-1)개 단어만 본다
 - bigram : $P(w_t | w_{(t-1)}) = \text{count}(w_{(t-1)}, w_t) / \text{count}(w_{(t-1)})$
 - trigram : $P(w_t | w_{(t-2)}, w_{(t-1)})$
- 마르코프 체인 텍스트 생성 = bigram 언어 모델에서 표본을 뽑는 것과 같음
- n이 커질수록 문맥은 좋아지지만 데이터 희소성(한 번도 못 본 n-gram) 문제가 커짐 → 스무딩 필요

6.2 텍스트 생성 (Text Generation)

- **마르코프 체인을 사용한 텍스트 생성 : 절차**
 - 1. 필요한 라이브러리를 임포트함
 - 2. 발화(speech)에서 얻은 텍스트를 정제할 함수를 추가함
 - 3. Gensim 라이브러리의 `strip_tags`, `strip_non_alphanumeric` 등 텍스트 정제 유틸리티를 사용함
 - 4. `load_speeches` 함수는 생성기(generator)이므로 나중에 소비 시점에 불러옴
 - 5. 발화에 있는 단어들을 담고 있는 리스트인 코퍼스를 불러올 함수를 정의함
 - 6. 연속해서 나오는 단어 쌍(bigram) 튜플들의 리스트를 만듦, 주어진 단어 뒤에 나올 단어를 생성하기 위해 이 단어 쌍을 사용함

6.2 텍스트 생성 (Text Generation)

- **마르코프 체인을 사용한 텍스트 생성 : 절차 (계속)**
 - 7. 함수들을 모아 파이프라인을 만듦 : speeches 로드 → corpus 생성 → 단어 dict 생성
 - 8. 두 종류의 코퍼스(예: 연설문, 코미디 대본)에 대해 각각 함수를 실행해 서로 다른 문체를 비교함
 - 9. 단어 dict로부터 문장을 만드는 get_sentence 함수를 만듦, 먼저 무작위로 시작 단어를 선택하고, 목표 문장 길이에 도달할 때까지 다음 단어를 확률적으로 선택함
 - 10. 각 코퍼스에서 샘플 문장을 생성해 문체의 차이를 관찰함

6.2 텍스트 생성 (Text Generation)

- 마르코프 체인 텍스트 생성 : 핵심 코드

- 단어 쌍으로 전이 사전(dict)을 만들고, 확률적으로 다음 단어를 선택함
- collections.defaultdict 를 쓰면 코드가 간결해짐

```
import random
from collections import defaultdict

def make_pairs(corpus):
    for i in range(len(corpus) - 1):
        yield corpus[i], corpus[i + 1]

def build_word_dict(corpus):
    d = defaultdict(list)
    for w1, w2 in make_pairs(corpus):
        d[w1].append(w2)
    return d
```

```
def get_sentence(d, n=20):
    w = random.choice(list(d.keys()))
    out = [w]
    for _ in range(n):
        if w not in d: break
        w = random.choice(d[w])
        out.append(w)
    return ' '.join(out)
```

6.2 텍스트 생성 (Text Generation)

◦ 마르코프 체인의 한계

- 문맥이 직전 한 단어(또는 $n-1$ 단어)뿐이라 문장 전체의 일관성이 없음
- 학습 데이터에 없던 단어 조합은 절대 생성하지 못함(일반화 불가)
- 단어를 '기호'로만 다루므로 의미의 유사성을 모름 - '고양이'와 '고양이들'을 전혀 다른 상태로 취급
- n 을 키우면 문맥은 좋아지지만 필요한 데이터가 지수적으로 증가함
- → 해결의 실마리 : 단어를 '연속적인 벡터'로 표현하고, 더 긴 문맥을 신경망으로 압축하기(6.3절)

6.2 텍스트 생성 (Text Generation)

◦ 신경망 언어 모델로의 발전

- NNLM (2003) : 단어를 임베딩 벡터로 바꾼 뒤 신경망으로 다음 단어를 예측 - 희소성 문제 완화
- RNN / LSTM / GRU : 은닉 상태로 임의 길이의 문맥을 누적 - 이론상 문맥 길이 제한 없음
 - 장점 : 순차적 정보를 자연스럽게 처리
 - 단점 : 순차 계산이라 병렬화가 어렵고, 아주 긴 의존 관계는 여전히 학습이 어려움
- Transformer (2017) : 셀프 어텐션으로 모든 위치를 한 번에 참조 - 병렬 학습 + 장거리 의존 처리

6.2 텍스트 생성 (Text Generation)

- **Transformer 기반 생성 모델 (GPT 계열)**

- 구조 : 트랜스포머 디코더만 쌓아 올린 자기회귀 (autoregressive) 언어 모델
- 학습 목표는 마르코프 체인과 본질적으로 같음 - '다음 토큰 예측'
 - 차이는 문맥의 길이(수천~수십만 토큰)와 표현의 방식(연속 벡터)임
- 토큰화 : 단어가 아니라 서브워드(BPE, SentencePiece) 단위를 사용해 미등록 단어 문제를 해결함

- **허깅페이스 예시**

```
from transformers import pipeline
gen = pipeline('text-generation', model='gpt2')
gen('Natural language processing is', max_length=50)
```

6.2 텍스트 생성 (Text Generation)

- 디코딩 전략 (1) : 결정적 방법

- 언어 모델은 매 시점 '다음 토큰의 확률 분포'를 출력함 - 여기서 무엇을 고를지가 디코딩 전략
- 그리디 탐색(greedy search)
 - 매 시점 확률이 가장 높은 토큰 하나를 선택
 - 빠르지만 지역 최적에 빠지고 같은 표현을 반복하기 쉬움
- 빔 탐색(beam search)
 - 확률이 높은 후보 k개(빔)를 동시에 유지하며 시퀀스 전체 확률을 최대화
 - 요약·번역처럼 정답이 비교적 정해진 과제에 적합
 - 창의적 생성에는 문장이 단조로워지는 단점이 있음

6.2 텍스트 생성 (Text Generation)

- 디코딩 전략 (2) : 확률적 샘플링

- 온도(temperature) T : 로짓을 T 로 나눠 분포의 날카로움을 조절
 - $T < 1 \rightarrow$ 분포가 뾰족해져 보수적, $T > 1 \rightarrow$ 평평해져 다양하고 위험함
- top-k 샘플링 : 확률 상위 k개 토큰으로 후보를 제한한 뒤 그 안에서 샘플링
- top-p (nucleus) 샘플링 : 누적 확률이 p가 될 때까지의 토큰만 후보로 삼음
 - 분포가 뾰족할 땐 후보가 적고, 평평할 땐 후보가 많아져 top-k보다 유연함
- 실무 기본값 예시 : temperature 0.7, top_p 0.9, repetition_penalty 1.1

6.2 텍스트 생성 (Text Generation)

◦ 생성 텍스트의 위험과 대응

- 환각(hallucination) : 사실이 아닌 내용을 그럴듯하게 생성 → 출처 기반 생성(RAG), 사실성 검증 단계 추가
- 반복(repetition) : 같은 구절을 되풀이 → repetition penalty, no_repeat_ngram_size 설정
- 편향(bias) : 학습 데이터의 사회적 편향을 그대로 재생산 → 데이터 정제, 평가 데이터셋으로 점검
- 저작권·개인정보 : 학습 데이터의 원문을 그대로 뱉을 수 있음 → 출력 필터링
- 교육적 시사점 : 생성 결과는 '초안'이며 사람의 검토가 전제되어야 함

6.3 벡터 표현 (Vector Representation)

- **학습 목표 :**

- 자연어 처리에서 벡터 표현이 필요한 이유
- 텍스트를 벡터로 표현하는 다양한 방법(문자/위치/원핫/BoW/TF-IDF)
- 단어 벡터와 단어 벡터의 다양한 형태(Word2Vec, GloVe, FastText)를 설명
- 벡터 연산(유사도, 유추)을 구현
- 문서 벡터와 문맥 임베딩을 설명

6.3 벡터 표현 (Vector Representation)

- 소개 :

- 언어가 어떻게 동작하고 인간과 컴퓨터 사이의 상호작용이 어떻게 발전해 왔는지에 대한 인사이트를 제공하는 핵심 주제를 다룸
- NLP의 핵심은 텍스트를 숫자로 표현하는 간단한 트릭임
- 이는 소프트웨어 알고리즘이 텍스트의 의미를 이해하게 해 주는 복잡한 계산을 수행하는 데 도움을 줌
- 텍스트 표현은 각 단어를 정수로 인코딩하는 것처럼 간단하게 할 수도 있고, 각 단어를 숫자들의 배열로 표현할 수도 있음
- 텍스트를 벡터로 인코딩하는 방법과, 반대로 벡터에서 의미를 추출하는 방법을 함께 살펴봄

6.3 벡터 표현 (Vector Representation)

- 벡터 정의 :

- 벡터는 실수 값들로 표현된 숫자의 집합임
- 벡터는 행렬에서 한 행으로 볼 수도 있음
- 텐서는 일반적인 개념으로 스칼라, 벡터, 행렬을 모두 포괄함
- 먼저 다음과 같은 텍스트의 단위에 대해서 살펴봄
 - 문자 : 알파벳의 한 단위
 - 단어 : 연속된 글자의 집합
 - 문장 : 연속된 단어들의 집합
 - 구절 : 단어들의 시퀀스
 - 문서 : 단어들의 집합이며 구절들로 나눌 수 있음

6.3 벡터 표현 (Vector Representation)

- 벡터 표현을 사용하는 이유 :

- 앞서 언급한 각 단위들은 형태가 서로 달라 NLP 알고리즘에서 그대로 다루기 어려움
- 데이터를 더 잘 이해하고 서로 다른 연산을 수행하기 위해 이 단위들을 벡터로 변환해야 함
- 컴퓨터는 기본적으로 0과 1만을 이해하므로, 화면에 표시되는 텍스트조차 일부 숫자 형식으로 인코딩됨
- NLP 알고리즘에는 많은 양의 텍스트 데이터가 필요함, 이 데이터를 처리하는 데 시간이 오래 걸리고 알고리즘 성능에 영향을 줌
- 데이터를 벡터로 표현하면 CPU/GPU가 데이터를 일괄(배치)적으로 처리할 수 있어 성능이 향상됨
- 또한 '유사도'라는 개념을 수학적으로 정의할 수 있게 됨

6.3 벡터 표현 (Vector Representation)

◦ 인코딩 전체 지도

- 인코딩 : 데이터를 특정 형식으로 변환하는 과정
 - 1. 문자 수준 인코딩
 - 2. 위치 기반 수준의 인코딩
 - 3. 원핫 인코딩
 - 4. 단어 수준 원핫 인코딩
 - 5. BoW / TF-IDF (빈도 기반 문서 표현)
 - 6. 단어 임베딩 (Word2Vec, GloVe, FastText)
 - 7. 문서 벡터 / 문맥 임베딩
- 위로 갈수록 희소(sparse)하고 의미를 담지 못하며, 아래로 갈수록 밀집(dense)하고 의미를 담음

6.3 벡터 표현 (Vector Representation)

- 인코딩 :

- 인코딩은 원래 데이터를 다른 프로세스에서 사용할 수 있는 형태로 바꾸기 위해 코드를 적용하는 과정임
- 텍스트가 포함된 파일에 가장 일반적으로 사용되는 인코딩 체계는 ASCII 이며, 오늘날에는 UTF-8 이 표준임
- 문자 수준 인코딩
 - 해당 문자에 대해 ASCII 코드(또는 유니코드 코드포인트)를 사용하는 것임
 - 예 : 'A' $\rightarrow$ 65, 'B' $\rightarrow$ 66
- 위치 기반 수준의 인코딩
 - 각 문자(또는 단어)를 그 위치에 따라 인코딩함
 - 해당 토큰이 최초로 나타나는 위치를 기준으로 정수 값을 부여해 만들 수 있음

6.3 벡터 표현 (Vector Representation)

◦ 원핫 인코딩

- 원핫 벡터는 어휘에 있는 각 단어를 나타내는 데 사용하는 $1 \times N$ 행렬임
- 이 벡터는 한 개의 셀에서만 1이 나오고, 나머지 모든 셀은 0으로 구성됨
- 원핫 인코딩의 주요 단계
 - 1. 인코딩할 문자나 단어를 결정함
 - 2. 텍스트를 토큰화할 방법과 프로젝트에 무관한 토큰을 제거할 방법을 결정함
 - 3. 문자나 단어의 총개수(고유 토큰 수 = 어휘 크기 N)를 결정함
 - 4. 각 토큰에 인덱스를 부여하고, 해당 인덱스만 1인 길이 N 벡터를 만듦

6.3 벡터 표현 (Vector Representation)

◦ 원핫 인코딩 예시

- 어휘 : ['나는', '밥을', '빵을', '먹었다'] $\rightarrow N = 4$
 - 나는 $\rightarrow [1, 0, 0, 0]$
 - 밥을 $\rightarrow [0, 1, 0, 0]$
 - 빵을 $\rightarrow [0, 0, 1, 0]$
 - 먹었다 $\rightarrow [0, 0, 0, 1]$
- '밥을'과 '빵을'의 코사인 유사도 = 0 - 두 단어가 얼마나 비슷한지 전혀 알 수 없음
- 어휘가 5만 개면 단어 하나가 5만 차원 벡터 - 메모리와 계산이 모두 낭비됨

6.3 벡터 표현 (Vector Representation)

◦ 단어 수준 원핫 인코딩

- 단어는 언어와 인간 언어에 담긴 의미의 기본 단위임
- 단어는 NLP 알고리즘이 텍스트에서 의미를 추출할 수 있는 가장 효율적인 벡터 표현 단위임
- 문자 수준에서 이뤄지는 대부분 유형의 표현은 단어 수준에서도 수행할 수 있음
- 단어에 대해서도 원핫 인코딩을 수행할 수 있음
- 다만 원핫 인코딩된 단어를 사용할 때는 어휘 개수에 주목해야 함, 어휘가 많으면 원핫 벡터의 크기 때문에 많은 메모리를 소모하게 됨

6.3 벡터 표현 (Vector Representation)

- **BoW(Bag of Words) 와 TF-IDF**

- BoW : 문서에 등장한 단어의 '개수'만 세어 문서를 하나의 벡터로 표현(어순은 버림)
 - 문서 D1 = '나는 밥을 먹었다', D2 = '나는 빵을 먹었다'
 - 어휘 = [나는, 밥을, 빵을, 먹었다] → D1 = [1,1,0,1], D2 = [1,0,1,1]
- 한계 : 모든 문서에 흔한 단어가 과대평가되고, 어순과 의미를 담지 못함
- TF-IDF : 흔한 단어의 가중치를 낮추어 '그 문서다움'을 강조함
 - $TF\text{-}IDF(t, d) = TF(t, d) \times [\log(N / (1 + df(t))) + 1]$
- 여전히 희소 표현이지만, 검색·분류에서는 지금도 강력한 기준선(baseline)임

6.3 벡터 표현 (Vector Representation)

- TF-IDF 계산 예시
- ※ 위 손계산은 6주차 판 공식이며, TfidfVectorizer 는 sklearn 기본식(+L2 정규화)이라 값이 다르다
 - 문서 3개, 단어 '자연어'가 문서 1에서 3회 등장, 문서 1의 총 단어 수 100
 - $TF = 3 / 100 = 0.03$
 - '자연어'가 등장한 문서 수 $df = 1 \rightarrow IDF = \log(3 / (1+1)) + 1 = \log(1.5) + 1 \approx 1.405$
 - $TF-IDF = 0.03 \times 1.405 \approx 0.0422$
 - 반면 '그리고'가 3개 문서 모두에 등장하면 보정항이 없을 때 $\log(3/(1+3)) \approx -0.29$ 로 음수가 되어 점수를 깎음 $\rightarrow +1$ 을 더해 $IDF \approx 0.71$ 로 작지만 양수가 되게 함
- **scikit-learn 구현**

```
from sklearn.feature_extraction.text import TfidfVectorizer
X = TfidfVectorizer().fit_transform(documents)
```

6.3 벡터 표현 (Vector Representation)

◦ 단어 임베딩

- 임베딩은 다른 구조를 안에 품고 있는 수학적 구조임
- 임베딩 벡터가 어떤 것의 값을 포함하거나 표현한다고 말할 수 있음
- 단어 임베딩은 단어가 벡터 공간에 임베딩되는 특수한 유형의 임베딩임
 - 변환 과정 : 단어 $\rightarrow$ 원핫 벡터 $\rightarrow$ (가중치 행렬 곱) $\rightarrow$ 임베딩 벡터
- 머신러닝 알고리즘에 활용하기 위해 텍스트를 벡터로 표현하는 방법을 찾아야 함
- 원핫 인코딩이 그중 한 가지 방법이지만, 그 한계를 해결할 방법이 필요함

6.3 벡터 표현 (Vector Representation)

- 원핫 인코딩의 한계와 임베딩의 목표
 - 원핫 벡터의 한계
 - 벡터가 상당히 커질 수 있음(어휘 크기 = 차원 수)
 - 벡터가 희소함(거의 모든 값이 0)
 - 벡터 사이의 상관관계 정보가 없음 - 두 벡터가 가까운지 먼지 결정할 수 없음
 - 따라서 목표는 다음 조건을 만족하는 단어 벡터 표현을 찾는 것임
 - 더 적은 차원으로 압축되어야 함(보통 50 ~ 300 차원)
 - 추가적인 의미 정보를 포함할 수 있어야 함
 - 각 단어 벡터에 다른 벡터와의 관계 정보가 포함되어야 함

6.3 벡터 표현 (Vector Representation)

- 임베딩이 담는 '의미' : 분포 가설

- 임베딩은 정보를 담고 있는 구조 - 단어의 의미를 함축하는 구조를 갖고 있다는 뜻임
- 각각의 임베딩 벡터는 한 단어의 의미에 대한 정보를 담고 있음
- 분포 가설(distributional hypothesis)
 - '단어의 의미는 그 주변에 함께 나타나는 단어들에 의해 결정된다' (Firth, 1957)
 - 따라서 같은 문맥에 나올 단어를 예측하도록 학습하면 의미를 담은 벡터를 얻을 수 있음
- 이 아이디어를 구현한 대표적인 알고리즘이 Word2Vec 임

6.3 벡터 표현 (Vector Representation)

- **Word2Vec : CBOW 와 Skip-gram**

- 단어 벡터를 학습하기 위해 개발한 알고리즘, 입력으로 텍스트 코퍼스를 받고 출력으로 단어 벡터를 만듦
- CBOW (Continuous Bag of Words)
 - 주변 단어들로 가운데 단어를 예측함
 - 학습이 빠르고 자주 등장하는 단어에 유리함
- Skip-gram
 - 가운데 단어로 주변 단어들을 예측함
 - 학습은 느리지만 희귀 단어의 표현이 더 좋음
- 윈도우 크기(window)는 문맥의 범위를 결정함 - 작으면 문법적, 크면 주제적 유사성을 학습함

6.3 벡터 표현 (Vector Representation)

- Word2Vec 학습 트릭과 구현

- 출력층 softmax 는 어휘 크기만큼 계산해야 해서 매우 느림
 - 네거티브 샘플링 : 정답 1개 + 무작위 오답 k개(보통 5~20)만으로 이진 분류 문제로 바꿔 학습
 - 서브샘플링 : 'the' 같은 초고빈도 단어를 확률적으로 제거해 학습 효율과 품질을 높임

- gensim 구현

```
from gensim.models import Word2Vec
model = Word2Vec(sentences, vector_size=100, window=5,
                  min_count=2, sg=1, negative=5, epochs=10)
model.wv.most_similar('tennis', topn=5)
```


6.3 벡터 표현 (Vector Representation)

- **GloVe 와 FastText**

- GloVe (Global Vectors)

- 전체 코퍼스의 단어 동시 등장 행렬(co-occurrence matrix)을 직접 분해해 학습
 - Word2Vec의 지역적 문맥 + 통계적 전역 정보를 결합한 방식
 - glove.6B.50d 처럼 사전 학습 벡터가 널리 공개되어 있어 TextRank 실습에 바로 사용함

- FastText

- 단어 앞뒤에 경계 기호를 붙인 뒤 문자 n-gram($n=3$)의 합으로 표현 : '<apple>' → <ap, app, ppl, ple, le>
 - 학습에 없던 단어(OOV)도 벡터를 만들 수 있고, 형태가 복잡한 언어(한국어)에 유리함

6.3 벡터 표현 (Vector Representation)

- 벡터 연산 : 유사도와 유추

- 유사도 : 코사인 유사도로 두 단어의 의미적 거리를 측정
 - $\cos(\text{king}, \text{queen})$ 은 높고, $\cos(\text{king}, \text{banana})$ 는 낮음
- 유추(analogy) : 벡터의 덧셈·뺄셈이 의미의 관계를 표현함
 - $\text{vec}(\text{king}) - \text{vec}(\text{man}) + \text{vec}(\text{woman}) \approx \text{vec}(\text{queen})$
 - $\text{vec}(\text{Paris}) - \text{vec}(\text{France}) + \text{vec}(\text{Korea}) \approx \text{vec}(\text{Seoul})$

- 구현

`model.wv.most_similar(positive=['king', 'woman'], negative=['man'])`

- 주의 : 학습 데이터의 사회적 편향도 그대로 벡터에 반영됨(직업-성별 편향 등)

6.3 벡터 표현 (Vector Representation)

◦ 사전 학습된 단어 벡터 사용

- 단어 벡터를 대규모 코퍼스로 학습해 두면 다른 관련 프로젝트에서도 저장하고 재사용할 수 있음
- 전이 학습(transfer learning)의 가장 단순한 형태임
- 다른 프로젝트에 활용하려면 다음 요인들을 고려해야 함
 - 학습 소스(도메인) : 위키피디아로 학습한 벡터를 의료 텍스트에 쓰면 성능이 떨어질 수 있음
 - 어휘 개수와 차원 수 : 50/100/200/300차원 중 과제와 자원에 맞게 선택
 - 사용한 문맥 윈도우 크기, 대소문자 처리 여부, 토큰화 방식의 일치
- 데이터가 충분하다면 도메인 코퍼스로 직접 학습하거나 미세조정하는 편이 좋음

6.3 벡터 표현 (Vector Representation)

◦ 문서 벡터

- 문서는 제품 리뷰, 트윗, 또는 영화 대사 모음으로 구성할 수 있음
- 몇 개의 단어부터 수천 개의 단어까지 다양한 길이를 가짐
- 가장 단순한 방법 : 문서에 포함된 단어 벡터의 평균(TextRank 실습에서 사용한 방식)
- 문서 벡터의 활용
 - 유사도 : 문서 벡터로 텍스트의 유사도를 비교할 수 있음, 예를 들어 법률 AI는 유사 판례를 찾는 데 사용함
 - 추천 : 온라인 잡지는 사용자가 이미 읽은 기사를 바탕으로 유사한 기사를 추천할 수 있음
 - 예측 : 문서 벡터를 분류·회귀 모델의 입력 특징으로 사용할 수 있음

6.3 벡터 표현 (Vector Representation)

- 더 나은 문서 벡터 : Doc2Vec 과 SBERT

- 단순 평균의 한계 : 어순을 무시하고, 중요한 단어와 사소한 단어를 똑같이 취급함
- Doc2Vec (Paragraph Vector) : 문서마다 고유한 벡터를 함께 학습해 문서 수준 의미를 담음
- SBERT (Sentence-BERT) : BERT를 문장 쌍 유사도로 미세조정해 문장 임베딩을 직접 생성
 - 의미 검색, 중복 질문 탐지, RAG의 검색 단계에서 표준적으로 사용됨

```
from sentence_transformers import SentenceTransformer  
emb = SentenceTransformer('all-MiniLM-L6-v2').encode(sentences)
```

6.3 벡터 표현 (Vector Representation)

◦ 정적 임베딩 vs 문맥 임베딩

- 정적(static) 임베딩 : Word2Vec, GloVe, FastText
 - 한 단어에 벡터가 하나뿐 - '배'(ship/pear/stomach)를 구분하지 못함(다의어 문제)
- 문맥(contextual) 임베딩 : ELMo, BERT, GPT
 - 같은 단어라도 문장에 따라 다른 벡터를 생성함
 - '강가에 배를 댔다'의 '배'와 '배가 아프다'의 '배'가 서로 다른 벡터가 됨
- 6주차의 흐름 : 원핫 → 빈도 기반(BoW/TF-IDF) → 정적 임베딩 → 문맥 임베딩
- 표현이 좋아질수록 요약과 생성의 품질도 함께 좋아진다는 점이 이번 주차의 핵심 메시지임

6.5 요약 (1/3)

◦ 문서 요약

- 자동 텍스트 요약의 개념과 이점(샘플링, 검색, 인덱싱, 읽기 시간, 질의응답)
- 텍스트 요약을 목적 / 입력 / 출력의 세 각도에서 살펴봄
- 추출적 텍스트 요약 : 단어 빈도, TF-IDF, 문장 유사도, 클러스터링 중심
- 추상적 텍스트 요약 : 시퀀스 투 시퀀스, 인코더-디코더, 어텐션, 트랜스포머, BART/T5
- TextRank : PageRank 기반의 비지도 추출 요약, 파이썬으로 전 과정 구현
- 평가 : ROUGE-1/2/L, BLEU, METEOR, BERTScore 와 사람 평가의 병행

6.5 요약 (2/3)

◦ 텍스트 생성

- 마르코프 체인 : 상태 전이 확률에 기반한 가장 단순한 생성 모델
- n-gram 언어 모델 : 연쇄 법칙의 근사, 데이터 희소성 문제
- 마르코프 체인의 한계 → 신경망 언어 모델(RNN/LSTM) → 트랜스포머(GPT)
- 디코딩 전략 : greedy, beam search, temperature, top-k, top-p
- 생성 텍스트의 위험 : 환각, 반복, 편향과 그 대응 방법

6.5 요약 (3/3)

◦ 벡터 표현

- 벡터 표현을 사용하는 이유 : 계산 가능성, 배치 처리, 유사도의 수학적 정의
- 인코딩 : 문자 수준, 위치 기반, 원핫, 단어 수준 원핫
- 빈도 기반 표현 : BoW, TF-IDF 와 그 한계
- 단어 임베딩 : 분포 가설, Word2Vec(CBOW/Skip-gram), 네거티브 샘플링, GloVe, FastText
- 벡터 연산 : 코사인 유사도와 유추($\text{king} - \text{man} + \text{woman} \approx \text{queen}$)
- 문서 벡터 : 단어 벡터 평균, Doc2Vec, SBERT / 정적 임베딩 vs 문맥 임베딩

참고 서적 및 자료

◦ 참고 서적

- 전창욱, 최태균, 조중현, 신성진, 『텐서플로 2와 머신러닝으로 시작하는 자연어 처리』, 위키북스
- 홉슨 레인, 하네스 막스 하프케, 콜 하워드, 『파이썬으로 배우는 자연어 처리 인 액션』, 제이펍
- 소햄 고시(Sohom Ghosh), 드와이트 거닝, 『예제로 배우는 자연어 처리 기초』, 에이콘
- 임희석, 『자연어처리 바이블 - 핵심이론, 응용시스템, 딥러닝』, 휴먼사이언스

◦ 참고 논문

- Mihalcea & Tarau (2004), TextRank: Bringing Order into Texts
- Mikolov et al. (2013), Efficient Estimation of Word Representations in Vector Space
- Vaswani et al. (2017), Attention Is All You Need
- Lin (2004), ROUGE: A Package for Automatic Evaluation of Summaries

질문 및 정리

